

Lab 4. Parameter Efficient Fine-Tuning

Энэхүү даалгаврын хүрээнд BERT хэлний моделийг full funetining хийх биш харин parameter efficient finetuning хийх юм. Parameter Efficient Fine-Tuning хийхдээ Hugging Face-н PEFT сан ашигласан.

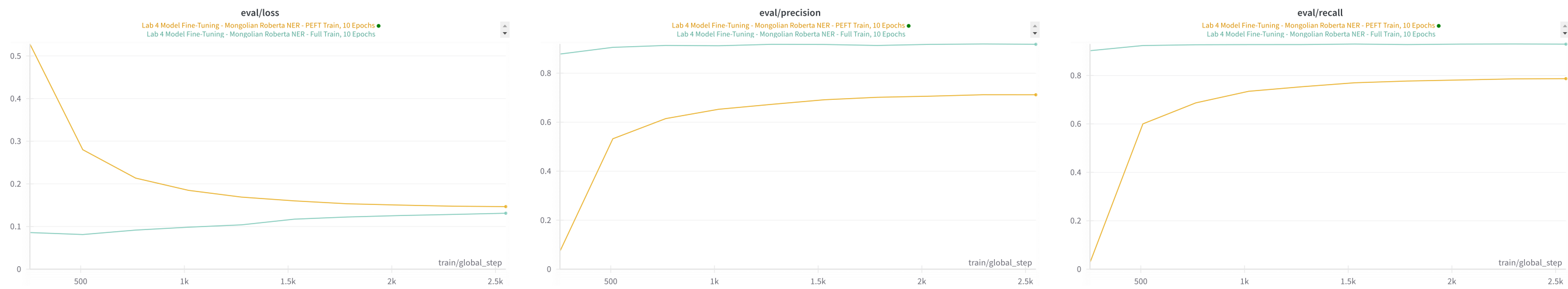
Дараах графт бүтэн сургалт болон хэсэгчилсэн сургалтын явцын үр дүнгүүдийг дүрсэллээ.



Дээрх графаас ажиглахад PEFT сургалтын градиентийн өөрчлөлт нь бүтэн сургалттай харьцуулахад илүү гөлгөр байгаа нь дараах шалтгаантай байж мэдэх юм:

1. PEFT нь моделийн бүх параметр биш харин хэсэг параметрт өөрчлөлт хийдэг тул градиент нь илүү цөөн удаа мөн илүү бага хэмжээгээр өөрчлөгдөх тул граф нь илүү гөлгөр байж болох юм.
2. PEFT нь нийт параметрийн дэд хэсэгтэй ажиллаж байгаа тул зорилгын функцийг локал орчинд оновчлох бодлого бодож байгаа тул градиентийн нормын граф нь бүтэн моделээс илүү гөлгөр байж магадгүй юм.

Дараах графт бүтэн болон хэсэгчилсэн сургалтын evaluation өгөгдөл дээрх явцын үр дүнгүүдийг дүрсэллээ.



```
In [ ]: import os
import json
from tqdm import tqdm
```

```

import numpy as np
import pandas as pd
from datasets import ClassLabel, load_dataset, Dataset, DatasetDict, load_metric

import torch
from transformers import (AutoConfig, AutoModelForTokenClassification, AutoTokenizer,
                          DataCollatorForTokenClassification, HfArgumentParser, PretrainedConfig,
                          PreTrainedTokenizerFast, Trainer, TrainingArguments, set_seed,)
from peft import (LoraConfig, get_peft_model, TaskType,
                  PeftModel, PeftConfig)

import warnings
warnings.filterwarnings('ignore')

import logging
logging.basicConfig(level = logging.INFO)
transformers_logger = logging.getLogger("transformers")
transformers_logger.setLevel(logging.WARNING)

from bayartsogtya_utils import dataset_prep

```

```

In [ ]: class CFG:
    wandb = True
    report_to = None
    lab_assignment = 3
    _wandb_kernel = "temuujin"

    debug = False
    num_workers = 12

    output_dir = "processed_data"
    model_save_dir = 'roberta-base-ner-demo'

    tokenizer_name = 'bayartsogt/mongolian-roberta-base'
    model_name = 'bayartsogt/mongolian-roberta-base'

    project = 'NUM-Machine-Learning-Lab-4'
    name = "Lab 4 Model Fine-Tuning - Mongolian Roberta NER - PEFT Train, 10 Epochs"

    config = {
        "output_dir": "mn_roberta_lab4_finetune_PEFT",
        "group": model_name,
        "learning_rate": 2e-5,
        "weight_decay": 1e-3,
        'num_train_epochs': 10,
        "train_batch_size": 32,
        "eval_batch_size": 32,
        "dataloader_num_workers": num_workers,
        "finetuning_task": 'ner',
        "evaluation_strategy": 'epoch',
        "logging_strategy": 'epoch',
        "overwrite_output_dir": True
    }

    test_size = 0.2

    train = True
    eval = True

```

```

eval_metric = "segeval"

early_stopping_patience = 15

if CFG.debug:
    CFG.config['num_train_epochs'] = 2

if CFG.wandb:
    os.environ["WANDB_SILENT"] = "True"
    CFG.report_to = "wandb"

    import wandb
    wandb.login()

    run = wandb.init(
        project = CFG.project,
        name = CFG.name,
        config = CFG.config
    )

config = CFG.config

```

1. Data prep

```

In [ ]: with open('./raw/NER_v1.0.json', 'r') as reader:
        lines = reader.readlines()
        lines = [json.loads(x) for x in lines]

raw_dataset, labels, label2idx, idx2label, num_labels = dataset_prep(lines)
raw_dataset

```

```

Out[ ]: Dataset({
  features: ['tokens', 'ner_tags'],
  num_rows: 10162
})

```

```

In [ ]: text_column_name = 'tokens'
        label_column_name = 'ner_tags'
        tokenizer = AutoTokenizer.from_pretrained(CFG.tokenizer_name, use_fast = True, add_prefix_space = True)

        # Tokenize all texts and align the labels with them.
        def tokenize_and_align_labels(examples):
            tokenized_inputs = tokenizer(examples[text_column_name],
                                         padding = "max_length",
                                         truncation = True,
                                         max_length = 128,
                                         is_split_into_words = True)

            labels = []
            for i, label in enumerate(examples[label_column_name]):
                word_ids = tokenized_inputs.word_ids(batch_index = i)
                previous_word_idx = None
                label_ids = []

                for word_idx in word_ids:
                    if word_idx is None:
                        label_ids.append(-100)
                    else:
                        label_ids.append(label[word_idx])

            return tokenized_inputs, labels

        tokenized_inputs, labels = tokenize_and_align_labels(examples)

```

```

        previous_word_idx = word_idx
        labels.append(label_ids)
        tokenized_inputs["labels"] = labels

    return tokenized_inputs

tokenized_dataset = raw_dataset.map(tokenize_and_align_labels, batched = True, num_proc = CFG.num_workers, desc = "Running tokenizer on train dataset")
all_dataset = tokenized_dataset.train_test_split(test_size = CFG.test_size)
all_dataset

```

Running tokenizer on train dataset (num_proc=12): 0%| | 0/10162 [00:00<?, ? examples/s]

```

Out[ ]: DatasetDict({
  train: Dataset({
    features: ['tokens', 'ner_tags', 'input_ids', 'attention_mask', 'labels'],
    num_rows: 8129
  })
  test: Dataset({
    features: ['tokens', 'ner_tags', 'input_ids', 'attention_mask', 'labels'],
    num_rows: 2033
  })
})

```

2. Evaluation metric

```

In [ ]: metric = load_metric(CFG.eval_metric)

def compute_metrics(p):
    predictions, labels = p
    predictions = np.argmax(predictions, axis=2)

    # Remove ignored index (special tokens)
    true_predictions = [
        [labels[p] for (p, l) in zip(prediction, label) if l != -100]
        for prediction, label in zip(predictions, labels)
    ]
    true_labels = [
        [labels[l] for (p, l) in zip(prediction, label) if l != -100]
        for prediction, label in zip(predictions, labels)
    ]

    results = metric.compute(predictions = true_predictions, references = true_labels)

    return {
        "precision": results["overall_precision"],
        "recall": results["overall_recall"],
        "f1": results["overall_f1"],
        "accuracy": results["overall_accuracy"]
    }

```

3. Training

```

In [ ]: lora_config = LoraConfig(
    task_type = TaskType.TOKEN_CLS,
    inference_mode = False,
    r = 8,

```

```

    lora_alpha = 32,
    lora_dropout = 0.3
)

auto_config = AutoConfig.from_pretrained(CFG.model_name,
                                         num_labels = num_labels,
                                         finetuning_task = config['finetuning_task'])

model = AutoModelForTokenClassification.from_pretrained(CFG.model_name, config = auto_config)
data_collator = DataCollatorForTokenClassification(tokenizer = tokenizer, pad_to_multiple_of = None)

model.config.label2id = label2idx
model.config.id2label = idx2label

peft_model = get_peft_model(model, lora_config)

peft_model.print_trainable_parameters()

```

Some weights of RobertaForTokenClassification were not initialized from the model checkpoint at bayartsogt/mongolian-roberta-base and are newly initialized: ['classifier.bias', 'classifier.weight']
 You should probably TRAIN this model on a down-stream task to be able to use it for predictions and inference.
 trainable params: 298,757 || all params: 124,357,642 || trainable%: 0.24024016151737582

```

In [ ]: test_ver = 2
        OUTPUT_MODEL = os.path.join(CFG.model_save_dir, f"test_v{test_ver}")

        training_args = TrainingArguments(
            report_to = CFG.report_to,
            output_dir = OUTPUT_MODEL,
            num_train_epochs = config["num_train_epochs"],
            per_device_train_batch_size = config["train_batch_size"],
            per_device_eval_batch_size = config["eval_batch_size"],
            overwrite_output_dir = config["overwrite_output_dir"],
            learning_rate = config["learning_rate"],
            weight_decay = config["weight_decay"],
            evaluation_strategy = config["evaluation_strategy"],
            do_eval = True,
            disable_tqdm = False
        )

```

```

In [ ]: trainer = Trainer(
    model = peft_model,
    args = training_args,
    train_dataset = all_dataset['train'],
    eval_dataset = all_dataset['test'],
    tokenizer = tokenizer,
    data_collator = data_collator,
    compute_metrics = compute_metrics,
)

```

```

In [ ]: torch.cuda.empty_cache()

```

```

trainer.train()

```

```

0%|          | 0/2550 [00:00<?, ?it/s]
0%|          | 0/64 [00:00<?, ?it/s]
{'eval_loss': 0.5280045866966248, 'eval_precision': 0.07313037723362012, 'eval_recall': 0.02752179327521793, 'eval_f1': 0.039992761491132824, 'eval_accuracy': 0.81168732222673
71, 'eval_runtime': 20.3069, 'eval_samples_per_second': 100.114, 'eval_steps_per_second': 3.152, 'epoch': 1.0}
{'loss': 0.6249, 'grad_norm': 1.5879558324813843, 'learning_rate': 1.607843137254902e-05, 'epoch': 1.96}

```

```
0%|          | 0/64 [00:00<?, ?it/s]
{'eval_loss': 0.28013721108436584, 'eval_precision': 0.5321921590281612, 'eval_recall': 0.6001245330012454, 'eval_f1': 0.5641205736025753, 'eval_accuracy': 0.9095895977245022,
'eval_runtime': 20.322, 'eval_samples_per_second': 100.039, 'eval_steps_per_second': 3.149, 'epoch': 2.0}
0%|          | 0/64 [00:00<?, ?it/s]
{'eval_loss': 0.2135339379310608, 'eval_precision': 0.6144846796657382, 'eval_recall': 0.686799501867995, 'eval_f1': 0.6486327550720375, 'eval_accuracy': 0.9286367330353514,
'eval_runtime': 20.136, 'eval_samples_per_second': 100.963, 'eval_steps_per_second': 3.178, 'epoch': 3.0}
{'loss': 0.26, 'grad_norm': 1.2867400646209717, 'learning_rate': 1.215686274509804e-05, 'epoch': 3.92}
0%|          | 0/64 [00:00<?, ?it/s]
{'eval_loss': 0.18481025099754333, 'eval_precision': 0.6525442477876107, 'eval_recall': 0.7346201743462017, 'eval_f1': 0.691154071470416, 'eval_accuracy': 0.9362301909792767,
'eval_runtime': 20.2861, 'eval_samples_per_second': 100.217, 'eval_steps_per_second': 3.155, 'epoch': 4.0}
0%|          | 0/64 [00:00<?, ?it/s]
{'eval_loss': 0.16901758313179016, 'eval_precision': 0.6725594841005115, 'eval_recall': 0.7533001245330012, 'eval_f1': 0.7106437969924813, 'eval_accuracy': 0.9406999187322227,
'eval_runtime': 20.2927, 'eval_samples_per_second': 100.184, 'eval_steps_per_second': 3.154, 'epoch': 5.0}
{'loss': 0.2048, 'grad_norm': 1.4882136583328247, 'learning_rate': 8.23529411764706e-06, 'epoch': 5.88}
0%|          | 0/64 [00:00<?, ?it/s]
{'eval_loss': 0.1601697951555252, 'eval_precision': 0.6913870246085011, 'eval_recall': 0.7697384806973848, 'eval_f1': 0.7284619917501473, 'eval_accuracy': 0.9438998374644454,
'eval_runtime': 18.5295, 'eval_samples_per_second': 109.717, 'eval_steps_per_second': 3.454, 'epoch': 6.0}
0%|          | 0/64 [00:00<?, ?it/s]
{'eval_loss': 0.15336984395980835, 'eval_precision': 0.7014052838673412, 'eval_recall': 0.7769613947696139, 'eval_f1': 0.7372525849335303, 'eval_accuracy': 0.9461854937017472,
'eval_runtime': 18.2509, 'eval_samples_per_second': 111.392, 'eval_steps_per_second': 3.507, 'epoch': 7.0}
{'loss': 0.1853, 'grad_norm': 1.7930268049240112, 'learning_rate': 4.313725490196079e-06, 'epoch': 7.84}
0%|          | 0/64 [00:00<?, ?it/s]
{'eval_loss': 0.1502869427204132, 'eval_precision': 0.7059750196916845, 'eval_recall': 0.7813200498132005, 'eval_f1': 0.7417390790329256, 'eval_accuracy': 0.9472775294595693,
'eval_runtime': 18.2551, 'eval_samples_per_second': 111.366, 'eval_steps_per_second': 3.506, 'epoch': 8.0}
0%|          | 0/64 [00:00<?, ?it/s]
{'eval_loss': 0.14767028391361237, 'eval_precision': 0.7118682310469314, 'eval_recall': 0.7858032378580324, 'eval_f1': 0.7470107730555227, 'eval_accuracy': 0.9482425843153189,
'eval_runtime': 18.1696, 'eval_samples_per_second': 111.89, 'eval_steps_per_second': 3.522, 'epoch': 9.0}
{'loss': 0.1767, 'grad_norm': 1.7384077310562134, 'learning_rate': 3.921568627450981e-07, 'epoch': 9.8}
0%|          | 0/64 [00:00<?, ?it/s]
{'eval_loss': 0.1466948240995407, 'eval_precision': 0.7117269347752619, 'eval_recall': 0.786799501867995, 'eval_f1': 0.74738274087656, 'eval_accuracy': 0.948623527021536, 'eval_runtime': 18.2895, 'eval_samples_per_second': 111.157, 'eval_steps_per_second': 3.499, 'epoch': 10.0}
{'train_runtime': 547.2505, 'train_samples_per_second': 148.543, 'train_steps_per_second': 4.66, 'train_loss': 0.28788456711114624, 'epoch': 10.0}
```

```
Out[ ]: TrainOutput(global_step=2550, training_loss=0.28788456711114624, metrics={'train_runtime': 547.2505, 'train_samples_per_second': 148.543, 'train_steps_per_second': 4.66, 'train_loss': 0.28788456711114624, 'epoch': 10.0})
```

```
In [ ]:
```